CSC1202, 2026, Week 6: Agent Memory

Part 1: Memory in LLM Agents

Part 2: Managing Memory

References. REF1. M. Grootendorst and J. Alammar (2026), An Illustrated Guide to AI Agents. O'Reilly Media.

Memory in LLM Agents

Why Do LLM Agents Need Memory?

LLMs are stateless: Each LLM call processes the provided prompt and information to generate a response. Previous prompts, responses and interactions are not automatically remembered.

Agents perform tasks across multiple steps and interactions. They need memory to retain and reuse relevant information to:

- **Maintain continuity:** Follow tasks across steps and sessions (e.g., resume refactoring the next day).
- **Personalise interactions:** Remember preferences (e.g., use pytest rather than unittest).
- **Support long-running tasks:** Remember what has already been done and what remains to be done, so the agent can continue without starting over.
- **Learn from experience:** Avoid repeating unsuccessful solutions (e.g., remember that a previous solution failed and try a different one).

What is agent memory ?

Agent memory enables an agent to retain and reuse information from previous interactions, actions and experiences across steps or sessions.

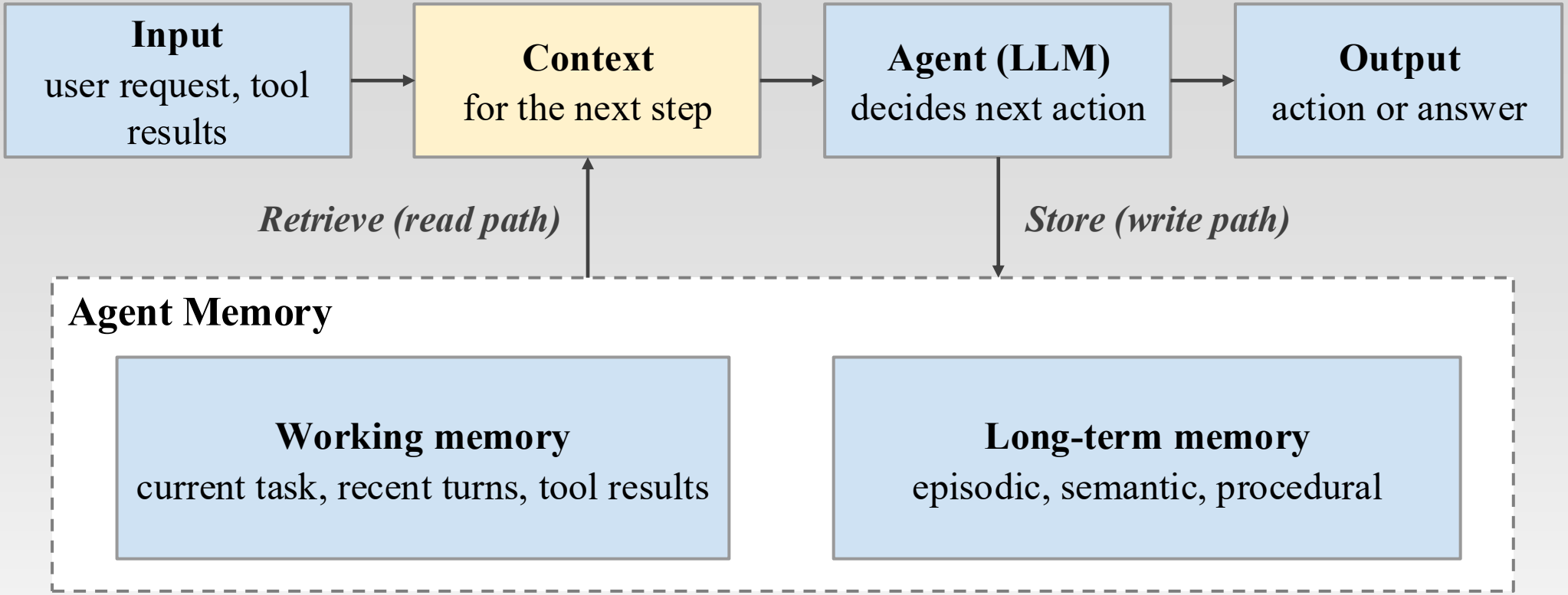
What can an agent remember?

- Knowledge from previous conversations (e.g., user preferences).
- Actions performed, their results and task progress.
- Decisions and experiences useful for future tasks.

Where is information held?

- **Context window (RAM analogy):** Information available to the LLM during the current call, including instructions and conversation history. Limited in size and not automatically retained between calls.
- **External memory (disk analogy):** Information stored in files, databases or vector stores for reuse across calls and sessions.
- **Model weights:** Knowledge and patterns learned during training, not normally updated through agent interactions.

Memory use



- **Writing to memory:** After a step / session, the agent may save information that may be useful later.

- **Retrieving from memory:** Before a subsequent step, the agent may retrieve relevant stored information and adds it to the LLM's context.

The LLM can use stored information only when it is included in the context provided to it.

Ref: Adapted from Alammar & Grootendorst, An Illustrated Guide to AI Agents, Figure 4-2.

Types of agent memory

Agent memory includes working memory and long-term memory:

Working memory: Information temporarily maintained by an agent during a session to support ongoing tasks (e.g., recent interactions, intermediate results, decisions and task progress).

Long-term memory: Information retained for future use. It includes:

- **Episodic memory (experiences):** Past events and interactions (e.g., remembering how an agent resolved a customer's complaint in a previous conversation).
- **Semantic memory (knowledge):** Information learned or retained about the world, users or tasks (e.g., a user's preferred programming language).
- **Procedural memory (procedures/workflows):** Stored instructions / procedures that guide how an agent performs tasks (e.g., the steps to follow when processing a customer refund).

Working memory

Working memory temporarily maintains information during an agent's session, including recent interactions, intermediate results, decisions and progress across tasks. It may be **cleared** when the session ends, while selected information can be **retained** in long-term memory for future reuse.

What does it contain?

- Current task goals and evolving plans, maintained across multiple steps (e.g., remembering the decision to exclude survey papers from further analysis).
- Intermediate results and observations from tool calls (e.g., retaining relevant findings from papers already retrieved).
- Actions completed and remaining steps (e.g., five papers reviewed, three still to examine).

Example: Research agent:

- The user asks the agent to investigate a research question.
- The agent searches for papers, examines abstracts and identifies relevant studies.
- As it progresses, working memory tracks papers already examined, key findings and papers still to examine.

Episodic memory (Experiences)

Episodic memory stores information about an agent's past experiences, including what happened, actions taken and their outcomes. It helps the agent reuse relevant experiences to inform future decisions and avoid repeating mistakes.

What does it contain?

- **Past tasks, interactions, problems encountered and their outcomes** (e.g., an agent recalls that a previous attempt to resolve a customer complaint failed because it misunderstood the request, but succeeded after asking for clarification).

Example: Customer support agent:

- A customer previously reported a billing error.
- The agent investigated the issue and arranged a refund.
- In a later interaction, the agent can recall what happened and how the issue was resolved.

Semantic memory (Knowledge)

Semantic memory stores knowledge that an agent can reuse across tasks and sessions, without needing to remember the specific experience in which the information was learned.

What does it contain?

- User information and preferences (e.g., a customer prefers communication by email).
- Facts about tasks or organisations (e.g., a company's standard refund period is 30 days).

Example: Customer support agent:

- The agent learns that a customer prefers email communication.
- It stores this preference for future interactions.
- When handling a new request, it retrieves the preference and uses it to communicate with the customer

Procedural Memory (Procedures)

Procedural memory stores reusable procedures/workflows that an agent can retrieve and follow when relevant to a given task.

What does it contain?

- Instructions and rules for performing tasks (e.g., a CLAUDE.md file specifying that automated tests must be run before changes are committed).
- Stored workflows (e.g., steps for changing a travel booking: check availability, confirm the new option and update the reservation).
- Tool-use procedures (e.g., instructions for invoking a reservation API to modify a booking).

Example: Travel booking agent

- The agent stores a procedure for changing reservations.
- When a customer requests a change, it retrieves and follows the stored procedure.
- The procedure can be updated based on experience and feedback (e.g., after a failed booking change, adding a step to check cancellation restrictions before proceeding).

All Four Memory Types in One Agent

- **Task:** A customer contacts an agent to request a refund for an order.
  - **Working memory:** During the current session, the agent tracks the order being discussed, information collected and progress of the refund request.
  - **Episodic memory:** The agent recalls a previous refund case where it initially misunderstood the customer's request and resolved the issue by asking for clarification.
  - **Semantic memory:** The company allows refunds within 30 days of purchase.
  - **Procedural memory:** The agent has a stored procedure for processing refunds: verify the order, check eligibility, obtain approval if required and initiate the refund.

Quiz 1

Task: Determine whether each statement is **True or False**.

1. An LLM automatically remembers previous interactions when the same agent makes a new LLM call.
2. Working memory can retain intermediate results and task progress during a multi-step task.
3. Information stored in external memory must be retrieved and included in the LLM's context before it can be used.
4. Episodic memory stores facts and preferences without necessarily retaining the experiences in which they were learned.
5. Procedural memory can include instructions for using tools and workflows for completing tasks.

Quiz 1: Answer

Answers: 1. False; 2. True; 3. True; 4. False; 5. True.

Quiz 2

Task: Label each situation as **Working, Episodic, Semantic or Procedural memory.**

1. While preparing a report, an agent keeps track of the sections it has completed and those it still needs to write.
2. An agent stores a record of a previous attempt to resolve a problem, including what it tried and why it failed.
3. An agent remembers that a research project uses PostgreSQL as its database.
4. An agent stores instructions describing the steps required to process a purchase request.

Quiz 2: Answer

Answers: 1. Working; 2. Episodic; 3. Semantic; 4. Procedural;

Managing Memory

Why Does Agent Memory Need Management?

Agent memory should not simply store everything.

Main challenges:

- **Irrelevant information:** Storing every interaction makes retrieval less effective.
- **Outdated information:** Previously stored information may no longer be correct (e.g., a customer changes their delivery address, but the agent still remembers the old address).
- **Privacy risks:** Storing sensitive information unnecessarily or retrieving it in an inappropriate context may expose private data (e.g., revealing one customer's account details to another customer).

Managing memory involves deciding what to store, how to organise it, what to retrieve, when to update it and what/when to remove.

How Agents Manage Memory

Agent memory is managed through operations that determine what information is retained, how it is maintained and when it is reused.

Main operations:

- **Capture and store:** Identify useful information from interactions, actions and observations (e.g., retaining a user's preferences).
- **Organise and consolidate:** Structure, combine or summarise related information (e.g., combining repeated observations into a concise memory).
- **Retrieve:** Find relevant stored information when needed (e.g., recalling a previous solution to a similar problem).
- **Update:** Revise stored information as new information becomes available (e.g., changing a customer's saved delivery address).
- **Remove:** Delete information that is outdated, irrelevant or should no longer be retained.

How is this implemented?

- **Built into the agent system:** Developers implement memory operations using tools and storage such as databases or files.
- **Using a memory framework:** Developers use an existing framework such as Mem0, which provides memory operations and storage integration.

Extracting Information for Memory

An agent uses an LLM to identify **information worth remembering** from interactions, actions and observations.

Example – Customer support agent

- **Customer:** "I prefer receiving updates by email rather than SMS."
- **Agent:** "I will use email for future updates.", **Customer:** "Can you check the status of my current order?"

Extracted memory information

- ''Customer prefers email communication rather than SMS.''

How extraction works

- The LLM identifies information potentially useful for future tasks and ignores information that does not need to be retained (e.g., the request to check the current order status).
- The LLM turns useful information into short, clear statements that can be understood and reused later (e.g., "Customer prefers email rather than SMS").

Consolidating Agent Memory

Consolidation compares newly extracted information with existing memories to **avoid duplicates and keep stored information up to date and conflict-free.**

Example – Customer support agent

- **Existing memory:** Customer prefers SMS notifications.
- **New information:** Customer says, "From now on, please send updates by email instead of SMS."
- **Consolidation:** Replace the old preference with **"Customer prefers email notifications."**

Possible consolidation decisions:

- **ADD:** Store new information that is not already remembered.
- **UPDATE:** Revise an existing memory when information changes
- **DELETE:** Remove an existing memory when it is found to be invalid or no longer needed.
- **NO CHANGE:** Keep the existing memory if the information is already stored.

Consolidation helps prevent duplicate, outdated or conflicting memories from affecting future interactions.

Forgetting and Compacting Memory

Agents need to **forget outdated or unnecessary memories** and **compact related memories** to avoid accumulating irrelevant or redundant information.

Forgetting: Reducing access to memories or removing them

- **Expiration:** Automatically delete a memory after a predefined period (e.g., a customer's temporary delivery address is stored for one week and then deleted).
- **Decay:** Reduce the priority of memories that have not been used for a long time (e.g., an old preference that is rarely relevant).
- **Explicit deletion:** : Remove a memory when it is no longer needed, is incorrect or irrelevant, or must be deleted at the user's request or to comply with a policy (e.g., a customer asks the agent to delete a previously stored delivery address).

Forgetting can be:

- **Soft:** Lower the priority of a memory or archive it without deleting it.
- **Hard:** Permanently remove the memory from storage.

Compaction: Reducing redundant stored information

- Merge duplicate or overlapping memory entries into a single consistent record (e.g., combine separate preferences for email updates, invoices and receipts into one email preference).

Trimming and Summarisation

Agents use **trimming and summarisation to reduce the size of memory** when the amount of information becomes too large.

Trimming – Removing selected information

- Shorten conversation history by removing messages or parts of messages that are unnecessary, redundant, or less relevant, rather than simply removing the oldest messages.
- Apply rules to determine what to remove (e.g., message relevance, repetition, or a limit on the size of the history).

Summarisation – Compressing information

- Replace detailed information with a shorter summary.
- Preserve important facts and decisions from the original information.

Example – Customer support agent: The agent has 20 stored memory entries about a customer's refund. These include the refund request, approval, payment status and repeated entries recording the same status.

- **Trimming:** Remove duplicate status entries, while keeping the refund request, approval and latest payment status.
- **Summarization:** Compress detailed stored information while preserving useful facts (e.g., replace a detailed record of a failed API call and its recovery with “API call failed due to missing authentication; succeeded after credentials were supplied”)

How Agents Retrieve Stored Memories

Agents use **different retrieval methods to find relevant information** from stored memories.

Common retrieval methods:

- **Keyword search:** Find memories containing specific keywords.
- **Semantic search:** Find memories with similar meaning, even when different words are used.
- **Metadata filtering:** Use stored attributes (e.g., customer ID or date) to limit which memories are searched.

Example – Customer support agent

A customer asks: “Has my replacement been sent yet?”, **Stored memory:** Stored memory: “Replacement laptop dispatched on Friday.” (Customer ID: 123)

Keyword search: May miss the memory because the customer says laptop rather than computer

Semantic search: Can identify the stored memory because the two expressions have similar meanings.

Metadata filtering: Restricts the search to memories associated with Customer ID 123, so the retrieved replacement status belongs to the correct customer.

Hot-Path vs Background Writing

Memory writing can happen during the current turn or asynchronously afterwards.

Hot-path writing

- Write memory before the agent returns its response.
- **Advantage:** The new memory is available immediately.
- **Cost:** Writing adds latency to the current response.

Background writing

- Write memory asynchronously after the response or session.
- **Advantage:** The current response does not wait for memory writing.
- **Cost:** The new memory is not available until writing completes.

Example — Customer support agent: A customer updates their preferred contact method from SMS to email.

- **Hot path:** Save the new preference before replying; subsequent actions can use it immediately.
- **Background:** Reply first and save the preference afterwards; another action occurring before the save completes may still use the old preference.

Quiz 3

Task: Determine whether each statement is **True or False**.

1. Memory extraction involves identifying information worth retaining from an interaction, rather than storing everything.
2. Consolidation means adding every newly extracted memory as a separate entry, even when it duplicates or contradicts existing information.
3. Updating memory may involve replacing an outdated preference when the user provides a new one.
4. Forgetting is only necessary when the memory storage capacity has been reached.
5. Compaction can combine several related memories into a shorter representation while preserving important information.

Quiz 3: Answer

Answers: 1. True; 2. False; 3. True; 4. False; 5. True.

Quiz 4

Task: Label each situation as **Extraction, Consolidation, Retrieval, or Compaction.**

1. An agent identifies a user's preference from a conversation and expresses it as a short statement for storage.
2. An agent compares newly extracted information with existing memories and decides to update an outdated record.
3. An agent searches its stored memories for information relevant to the current request.
4. An agent combines several related memories into a shorter representation while preserving important information.
5. An agent examines a conversation and selects a fact that may be useful in future interactions.

Quiz 4: Answer

Answers: 1. Extraction; 2. Consolidation; 3. Retrieval; 4. Compaction; 5. Extraction.

Summary: Agent Memory

- **Memory:** Enables agents to retain and reuse information across tasks and interactions, including facts, experiences and procedures.

- **Memory management:** Involves capturing, storing, organising, retrieving, updating and removing information.